

Javascript

DOM

Bjarte Wang-Kileng

HVL

19. september, 2026



**Western Norway
University of
Applied Sciences**

Outline

- 1 Demo *Terning*
- 2 Representere HTML elementer i JavaScript
- 3 Modifisere HTML innhold
- 4 HTML attributter
- 5 FORM input elementer
- 6 CSS

JavaScript og operasjoner på HTML elementer

- ▶ Lese, legge til, slette og modifisere HTML innhold.
- ▶ Også operasjoner på CSS stil-ark.

Adressere HTML element fra JavaScript

Før operasjon på web element må JavaScript finne elementet.

Metoder for å finne element returnerer objekt som representerer elementet i JavaScript.

HTML innhold

► Tekst-innhold i HTML element:

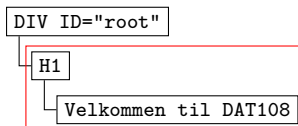
```
<!-- Tekst i et DIV (og H1 element) -->
<div id="root">
  <h1>Velkommen til DAT108</h1>
</div>
```

► HTML element attributt.

```
<!-- Attribute til et DIV element -->
<div id="root">
  <h1>Velkommen til DAT108</h1>
</div>
```

► HTML tre-struktur:

```
<!-- Tre struktur nedenfor en DIV -->
<div id="root">
  <h1>Velkommen til DAT108</h1>
</div>
```



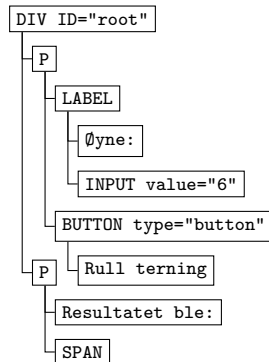
Outline

- 1 Demo *Terning*
- 2 Representere HTML elementer i JavaScript
- 3 Modifisere HTML innhold
- 4 HTML attributter
- 5 FORM input elementer
- 6 CSS

Demo med webside

- ▶ Se vedlagt arkiv med demonstrasjon på Canvas.
- ▶ HTML tre-struktur:

```
<DIV id="root">
  <P>
    <LABEL>Øyne: <INPUT value="6" ...></LABEL>
    <BUTTON type="button">Rull terning</BUTTON>
  </P>
  <P>Resultatet ble: <SPAN></SPAN></P>
</DIV>
```



- ▶ Webside:

Terning

Øyne:

Resultatet ble:

HTML element og JavaScript objekter

▶ JS element for DIV-elementet:

```
const rootElement = document.getElementById("root");
```

▶ JS element for INPUT-elementet:

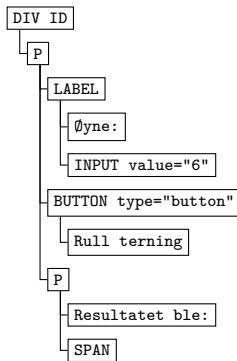
```
const inputElement = rootElement.querySelector("INPUT");
```

▶ JS element for BUTTON-elementet:

```
const buttonElement = rootElement.querySelector("BUTTON");
```

▶ JS element for SPAN-elementet:

```
const spanElement = rootElement.querySelector("SPAN");
```



Terning

Øyne:

Resultatet ble:

INPUT elementet sin verdi

- ▶ Verdi som vises i INPUT elementet er gitt av attributtet *value*.
- ▶ HTML attributt *value* er tilgjengelig som objekt-egenskap *value*.
 - HTML INPUT element med attributt *value*:

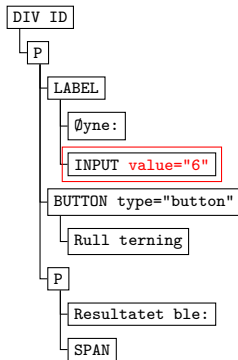
```
<INPUT value="6" ...>
```

- JavaScript objekt for INPUT element har egenskap *value*:

```
const maksverdi = inputElement.value;
```

- ▶ JS kode som endrerer *value*:

```
inputElement.value = "12";
```



Terning

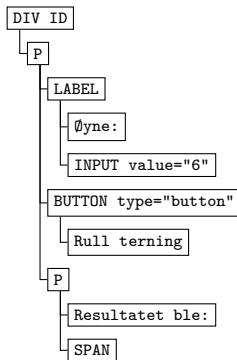
Øyne:

Resultatet ble:

Vise resultat i SPAN elementet

- ▶ JS objekt-egenskap *textContent* er tekst-innholdet summert over alle barn til et HTML-element.
- ▶ Egenskapen kan både leses og endres.
- ▶ JS kode for å vise teksten «4» i SPAN elementet på websiden:

```
spanElement.textContent = "4";
```



Terning

Øyne: 6

Resultatet ble:

Lytter for klikk hendelse på BUTTON elementet

- ▶ JS objekt metode *addEventListener* legger på lytter for hendelse.

- Hendelse kan f.eks. være klikk med musen.

- ▶ Legge på lytter for **click** på BUTTON:

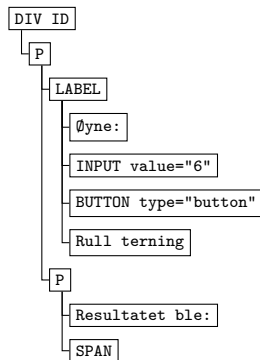
```
buttonElement.addEventListener("click", rullterning);
```

- ▶ Hendelse **click** skjer når bruker klikker med venstre musetast.

- ▶ Parameter *rullterning* er navnet på en funksjon.

- Blir ikke kjørt her da ingen parenteser () etter navnet.

- ▶ Funksjonen blir kjørt ved klikk på BUTTON.



Terning

Øyne:

Resultatet ble:

Outline

- 1 Demo *Terning*
- 2 Representere HTML elementer i JavaScript
- 3 Modifisere HTML innhold
- 4 HTML attributter
- 5 FORM input elementer
- 6 CSS

Absolutte og relative dokument-stier

- ▶ Absolutte sti tar utgangspunkt i *document*.

```
const rootElement = document.querySelector("BUTTON");
```

- ▶ Relative sti tar utgangspunkt i et annet web-element.

```
const submitButton = rootElement.querySelector("BUTTON");
```

- ▶ Absolutte stier gjør at koden ikke er portabel.
- ▶ Absolutte stier kan lage problemer ved endringer i dokumentet.

Absolutt element-stier

- ▶ Vi vil kun trenge metoden `document.getElementById`.
- ▶ Metoder for relative stier kan også brukes på *document*.
 - Gir da absolutte stier.
- ▶ Finnes diverse *document* egenskaper som er absolutte stier.
 - F.eks. *document.body* og *document.documentElement*.

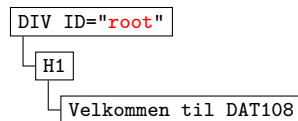
Metode *getElementById*

- ▶ Absolutt sti til HTML element.
- ▶ Returnerer objekt som representerer HTML element med et gitt HTML ID attributt.
- ▶ Returnerer **null** hvis ikke HTML elementet finnes.
- ▶ Observer at en HTML ID er unik i et HTML-dokument.

Demo med *getElementById*

► HTML:

```
<div id="root">  
  <h1>  
    Velkommen til DAT108  
  </h1>  
</div>
```



► JavaScript:

```
const rootElement = document.getElementById("root");
```

Bruk av relative stier til HTML elementer

- ▶ Adresseres med utgangspunkt i et annet HTML-element.
- ▶ Kan finne element utifra tagg, foreldre/søsken/barn relasjoner, *class* attributtet, andre HTML-attributter og attributt-verdier.
- ▶ HTML 5 tillater egendefinerte attributter som starter med «data-».

```
<TABLE>
  <TR><th>Navn</th><th>Telefonnummer</th></TR>
  <TR data-StudentID="5"><TD>Ole</TD><TD>12345678</TD></TR>
  <TR data-StudentID="9"><TD>Ane</TD><TD>87654321</TD></TR>
</TABLE>
```

JavaScript objekter fra relative stier til HTML elementer

- ▶ Finne element utifra egenskap ved element:
 - F.eks. *querySelector()*, *getElementsByTagName()*, *getElementsByClassName()*, *querySelectorAll()*.
- ▶ Mange elementer har egne tilpassede egenskaper, f.eks.:
 - HTML *TABLE*, *THEAD* og *TBODY* har egenskap *rows*.
 - HTML tabellrekker, tagg *TR* har egenskap *cells*.
- ▶ Finne element utifra relasjon:
 - F.eks. *parentNode*, *children*, *firstElementChild*, *lastElementChild*, *previousElementSibling*, *nextElementSibling*.

Metode *querySelector*

- ▶ Returnerer HTML element som stemmer med en gitt CSS selektor.
- ▶ Hvis flere HTML elementer passer til selektor returneres første.
- ▶ Returnerer **null** hvis ingen elementer stemmer med selektor.
- ▶ Gir feil **DOMException** ved ugyldig CSS-selector.
- ▶ Tidligere HVL kurs skal ha gitt en introduksjon til CSS-selektorer.

Demo med *querySelector*

► HTML:

```
<div id="root">  
  <h1>  
    Velkommen til DAT108  
  </h1>  
</div>
```

► JavaScript - *querySelector* som en *document* metode:

```
const element= document.querySelector("h1:first-child");
```

► JavaScript - *querySelector* som en *element* metode:

```
const rootElement = document.getElementById("root");  
const element= rootElement.querySelector("h1:first-child");
```

querySelectorAll()

- ▶ Returnerer liste av alle elementer som stemmer med selektor.
- ▶ Listen er tom hvis ingen elementer stemmer med selektor.
- ▶ Gir feil **DOMException** ved ugyldig CSS-selector.

Demo med *querySelectorAll*

► HTML:

```
<table id="root">
  <tbody>
    <tr><td>Ole</td><td>Olsen</td></tr>
  </tbody>
</table>
```

► JavaScript - *querySelectorAll* som en *document* metode:

```
const elements= document.querySelectorAll("tbody *");
console.log(`TBODY inneholder ${elements.length} tagger`);
```

► JavaScript - *querySelectorAll* som en *element* metode:

```
const rootElement = document.getElementById("root");
const elements = rootElement.querySelectorAll("tbody *");
console.log(`TBODY inneholder ${elements.length} tagger`);
```

getElementsByTagName()

- ▶ Returnerer en liste av HTML-elementer med gitt tagg.
- ▶ Argument «*» gir en liste av alle elementer som har tagg.
 - Tekst, attributter og kommentarer er også elementer, men uten tagg.
- ▶ Alle elementer i strukturen under *rootElement* med tagg *TABLE*:

```
const tableElements = rootElement.getElementsByTagName("TABLE");
```

- ▶ Alle elementer i strukturen under *rootElement* uansett tagg:

```
const tableElements = rootElement.getElementsByTagName("*");
```

- ▶ Første element under *rootElement* med tagg *BUTTON*:

```
const buttonRef = rootElement.getElementsByTagName("BUTTON")[0];
```

getElementsByClassName()

- ▶ Returnerer en liste av HTML-elementer med gitt *class* attributt.

```
<DIV id="root">  
  <P class="firstname">Ole</P>  
  <P class="firstname">Anne</P>  
</DIV>
```

- ▶ Liste av alle elementer under *rootElement* med *class* attributt **firstname**:

```
const elements = rootElement.getElementsByClassName("firstname");
```

- ▶ Andre element under *rootElement* med *class* attributt **firstname**:

```
const elements = rootElement.getElementsByClassName("firstname")[1];
```

- ▶ Antall elementer under *rootElement* med *class* attributt **firstname**:

```
const count = rootElement.getElementsByClassName("firstname").length;
```

Lister av web-elementer

- ▶ Flere DOM metoder returnerer lister med elementer, f.eks. *getElementsByTagName()*.
- ▶ Listene er vanligvis «levende».
 - Men *querySelectorAll()* returnerer en statisk liste.
- ▶ En levende liste oppdateres hvis dokumentstrukturen endres.
 - Listen vil hele tiden reflektere websiden slik den er akkurat nå, selv om listen ble opprettet tidligere.
- ▶ Listene kan bare oppdateres ved å endre på web-dokumentet.

Elementlister i *Prototype* og *jQuery*

- ▶ Rammeverk som *Prototype* og *jQuery* inkluderer metoder som returnerer lister av web-elementer.
- ▶ Disse listene implementeres som forekomster av JavaScript **Array**.
- ▶ Disse listene er statiske. Endringer i dokumentet vil ikke forplante seg til eksisterende lister.

DOM elementer som element egenskaper

- ▶ Elementer kan ha egenskaper og metoder tilpasset til elementet.
- ▶ Kan alltid bruke de generelle metodene, men de tilpassede metodene kan gi enklere kode.
- ▶ Sjekk dokumentasjonen for det enkelte HTML element.

Eksempler på element-egenskaper for HTML elementer

- ▶ HTML *TABLE* har egenskaper *tHead*, *rows* og *tBodies*.
 - *tHead* er tabellen sin *THEAD*.
 - *tBodies* er en levende liste av alle *TBODY* elementer.
 - *rows* er en levende liste av alle tabellrekkene i tabellen.
- ▶ HTML *TBODY* og *THEAD* har egenskap *rows*.
 - Levende liste av alle tabellrekkene.
- ▶ Tabellrekker, tagg *TR* har egenskap *cells*
 - Levende liste av alle tabellcellene.
- ▶ HTML *SELECT* har en egenskap *options*.
 - Levende liste av alle options-elementene.

Egenskaper for barn-foreldre relasjoner

```
<FORM action="behandledate" method="POST">  
  <LABEL>Navn: <INPUT type="text"></LABEL>  
  <BUTTON type="submit">Send</BUTTON>  
  <BUTTON type="reset">Nullstill</BUTTON>  
</FORM>
```

- ▶ Egenskapen *children* er en levende liste av alle barn med tagg.
 - FORM elementet har tre *children*, et *INPUT* og to *BUTTON*.
- ▶ *firstElementChild* er første barn i listen av barneelementer.
 - FORM elementet sitt *firstElementChild* er elementet *INPUT*.
- ▶ *lastElementChild* er siste barn i listen av barneelementer.
 - FORM elementet sitt *lastElementChild* er reset-knappen.
- ▶ *parentNode* er elementet sitt forelder element.

Egenskaper for søsken relasjoner

```
<FORM action="behandledate" method="POST">  
  <LABEL>Navn: <INPUT type="text"></LABEL>  
  <BUTTON type="submit">Send</BUTTON>  
  <BUTTON type="reset">Nullstill</BUTTON>  
</FORM>
```

- ▶ *previousElementSibling* er søskenelementet foran.
 - Reset-knappen sin *previousElementSibling* er submit-knappen.
- ▶ *nextElementSibling* er søskenelementet etter.
 - Submit-knappen sin *nextElementSibling* er reset-knappen.

Outline

- 1 Demo *Terning*
- 2 Representere HTML elementer i JavaScript
- 3 Modifisere HTML innhold
- 4 HTML attributter
- 5 FORM input elementer
- 6 CSS

Modifisere dokument

- ▶ *innerText*, *textContent* og *insertAdjacentText()* kan endre på tekst som vises i web-dokumentet.
- ▶ *innerHTML*, *outerHTML*, *insertAdjacentHTML()*, *setHTML()* og *setHTMLUnsafe()* kan endre på dokumentstrukturen til web-dokumentet.
 - Metode *setHTML()* er tilgjengelig i flere nettlesere fra våren 2026.
- ▶ *classList* kan endre verdier i et HTML element sitt attributt *class*.

```
<SPAN class="fornavn student">01e</SPAN>
```

- *SPAN* elementet har to verdier i *class* attributtet: *fornavn* og *student*.

Flere metoder for å modifisere dokument

- ▶ HTML elementer kan ha egne metoder for å arbeide med sine barn.
 - F.eks. har HTML sitt *TABLE* element metoder for å legge til og slette tabellrekker.
- ▶ Finnes også mer grunnleggende metoder for å lage HTML-elementer, f.eks. **Document** sine metoder `createElement()` og `createTextNode()`.

innerText og *textContent*

- ▶ *innerText* returnerer synlig tekst i element.
- ▶ *textContent* returnerer all tekst i element.
- ▶ Eksempel der *innerText* og *textContent* gir ulike svar:

```
<P>Hei <SPAN style="display:none">dere</SPAN></P>
```

- ▶ Kan både lese og endre tekstinnhold.

```
root.innerText = "Adjø";
```

- ▶ Kun ren tekst, ikke HTML kan legges til element.
 - Tilsvarende egenskaper for HTML kan gi sikkerhetshull.
 - Forsiktig med *innerHTML*, *outerHTML*, *insertAdjacentHTML()* og *setHTMLUnsafe()*.

innerHTML, *outerHTML*, *insertAdjacentHTML()*, *setHTML()* og *setHTMLUnsafe()*

- ▶ Disse metodene lar oss enkelt opprette nye HTML-strukturer.
- ▶ Eksempelet under viser riktig bruk av *insertAdjacentHTML*:

```
function showMessage(message) {  
  const root = document.querySelector("DIV[data-messages]");  
  root.insertAdjacentHTML('beforeend', "<P></P>");  
  root.lastElementChild.textContent = message;  
}
```

- ▶ *insertAdjacentHTML()* kan være mye raskere enn *innerHTML*.

HTML metoder og XSS

- ▶ Disse metodene må **aldri** brukes på brukerdata.
- ▶ Følgende må aldri gjøres:

```
function showMessage(message) {  
  const root = document.querySelector("DIV[data-messages]");  
  root.insertAdjacentHTML('beforeend', `<P>${message}</P>`);  
}
```

- ▶ Koden i siste eksempel gjør applikasjonen sårbar for **XSS** angrep.
- ▶ *innerHTML*, *outerHTML* og *setHTMLUnsafe* er like problematiske.
 - Metode *setHTML* er sikker, men kun tilgjengelig fra våren 2026.
- ▶ Koden er sårbar for injeksjon av HTML og CSS.
- ▶ Injeksjon av JavaScript er mulig f.eks. via attributt *onError*.

Tekst-maler (*Template strings*)

- ▶ Eksempelet på forrige side viser bruk av en tekst-mal.
- ▶ En tekst-mal erstatter variabler i mal med verdier.
- ▶ Et eksempel i JavaScript:

```
const student = {"navn": "Ole Olsen", "id": 134};  
const studentInfo = `${student.navn} har id ${student.id}`;
```

- ▶ JavaScript setter inn verdier fra variabler når mal blir definert.
 - Malen blir ikke forandret om variabler blir endret senere.
- ▶ For mer informasjon se f.eks. [Template literals](#).

Outline

- 1 Demo *Terning*
- 2 Representere HTML elementer i JavaScript
- 3 Modifisere HTML innhold
- 4 HTML attributter**
- 5 FORM input elementer
- 6 CSS

HTML attributter speilet i JavaScript

- ▶ JavaScript objekt-egenskaper kan speile HTML attributter.
- ▶ HTML:

```
<A href="https://www.hvl.no/">Besøk HVL</A>
```

- ▶ JavaScript:

```
const anchor = document.querySelector("A");  
anchor.href = "https://www.hvl.no/";  
anchor.textContent = "Besøk HVL";
```

- ▶ Observer at ikke alle attributter speiles.

Element metoder for å aksessere HTML attributter

- ▶ Element metode `getAttribute` for å returnere attributt-verdi.

```
const anchor = document.querySelector("A");
console.log(`Har href ${anchor.getAttribute("href")}`);
```

- ▶ Element metode `setAttribute` for å sette attributt-verdi.

```
const anchor = document.querySelector("A");
anchor.setAttribute("href", "https://www.hvl.no/");
```

- ▶ Element metode `hasAttribute` for å sjekke om element har attributt.

```
if (anchor.hasAttribute("href")) {
  console.log("Anker-elementet har href attributt");
}
```

Egendefinerte data attributter

- ▶ Brukerdefinerte HTML attributter må starte med “data-”

```
<P data-kurs='DAT152'>Velkommen til kurset.</P>
```

- ▶ Attributter på formen “data-” kalles [custom data attributes](#).
- ▶ Egenskapen [dataset](#) gir lese- og skrivetilgang til “data-” attributter.

```
const pelm = document.querySelector("P");  
if (pelm.dataset.kurs !== undefined) {  
    console.log(`Kurskode er ${pelm.dataset.kurs}`);  
}  
  
const nyttkurs = 'DAT151';  
pelm.dataset.kurs = nyttkurs;
```

- ▶ Observer at “data-” delen er fjernet fra attributt navnet.
 - For flere regler for navn på egenskaper, se f.eks. [dataset property](#).

Outline

- 1 Demo *Terning*
- 2 Representere HTML elementer i JavaScript
- 3 Modifisere HTML innhold
- 4 HTML attributter
- 5 FORM input elementer
- 6 CSS

HTML FORM input elementer

- ▶ FORM elementer er ment for input fra bruker og brukerhendelser.
- ▶ **Input elementer** lar bruker gi input data til applikasjonen.
- ▶ **Button elementet** er ment for å håndtere klikk med musen.
- ▶ Bruker-data til input element er tilgjengelig som egenskapen *value*.
 - **HTML attributet *value*** er den opprinnelige verdien til elementet.
 - **JS egenskapen *value*** er den nåværende verdien til elementet.

Demo med FORM input elementer

► HTML:

```
<h1>Velkommen til <span>kurset</span></h1>

<fieldset>
  <legend>Fyll inn navn på kurset</legend>
  <label>Kurs: <input type="text" placeholder="Navn på kurs"></label>
  <button type="button">Registrer navn på kurs</button>
</fieldset>
```

► JavaScript:

```
function registrerkurs() {
  const inputElement = document.querySelector("input[type=text]");
  const kurs = inputElement.value.trim();
  if (kurs === "") return;

  const spanElement = document.querySelector("h1").querySelector("span");
  spanElement.textContent = kurs;
}

const button = document.querySelector("button");
button.addEventListener("click", registrerkurs);
```

Data fra FORM input elementer

- ▶ Attributt *value* er verdien til input elementet som tekst.
- ▶ Noen INPUT elementer har også en JS egenskap *valueAsNumber*.
 - Hvis verdi ikke er tall returneres verdi **NaN**.
- ▶ Tallverdi fra *valueAsNumber* avhenger av type INPUT element:
 - *number* og *range*: Tallet som er fylt inn.
 - *date* og *datetime-local*: Millisekunder siden midnatt 1. januar 1970.
 - *time*: Millisekunder siden midnatt samme dag.
 - *month*: Antall måneder siden 1. januar 1970.
 - Ikke Firefox eller Safari.
 - *week*: Millisekunder fra 1. januar 1970 til første dag denne uken.
 - Ikke Firefox eller Safari.
- ▶ INPUT for dato og tid har også JS egenskap *valueAsDate*.
 - Gjelder ikke INPUT for *datetime-local*.

Outline

- 1 Demo *Terning*
- 2 Representere HTML elementer i JavaScript
- 3 Modifisere HTML innhold
- 4 HTML attributter
- 5 FORM input elementer
- 6 CSS

Element egenskap *classList*

- ▶ Egenskapen *classList* gir tilgang til CSS klasser til element.
- ▶ Et HTML element med to CSS class verdier **fornavn** and **student**:

```
<SPAN class="fornavn student">Olle</SPAN>
```

- ▶ Hvert CSS class verdi svarer til et class name objekt i *classList*.
- ▶ Egenskapen er en **levende liste** med alle class name objekter.
- ▶ Inkluderer metoder for å arbeide med listen av CSS class verdier.

Modifisere utseende til HTML elementer

Anbefalt tilnærming i JavaScript for å modifisere utseende til HTML elementer bruker *classList* for å legge til eller fjerne CSS class verdier.

Arbeide med *classList*

- ▶ Egenskap *length* er antall class verdier til HTML elementet.

```
const element = document.querySelector("p");  
console.log(`Element has ${element.classList.length} class names`);
```

- ▶ Metoden *toggle* alternerer i å legge til og fjerne class verdier.
 - Returner **true** hvis class verdi ble lagt HTML element.
- ▶ Metode *contains* sjekker om HTML element har et class name objekt.
- ▶ Metode *remove* fjerner et class name objekt fra HTML element.
- ▶ Metode *add* legger til class verdi to HTML element.
- ▶ For alle metoder, se [Element: classList property](#) og [DOMTokenList](#).